Temas de Proyectos para Simular en la clase de Arquitectura de Sistemas Operativos

El sistema operativo se sitúa entre tu hardware (RAM, CPU, disco duro) y tu software (Chrome, VS Code, Spotify). Gestiona seis aspectos principales:

1. **Gestión de memoria**: decide qué programas obtienen cuánta RAM y gestiona la memoria virtual.
2. **Gestión de procesos**: programa qué programas se ejecutan y cuándo, gestiona la multitarea.
3. **Sistema de archivos**: organiza cómo se almacenan, leen y eliminan los datos en el disco.
4. **Controladores de dispositivos**: permiten que el sistema operativo se comunique con el hardware, como el teclado, la GPU y la impresora.
5. **Asignación de recursos**: comparte el tiempo de CPU y la memoria de manera equitativa entre los programas.
6. **Seguridad**: gestiona los permisos de usuario, las contraseñas y el control de acceso.

Puede desarrollarse en cualquiera de los siguientes lenguajes:

- **C# (.NET)** (Recomendado por su facilidad para crear interfaces gráficas en Windows y acceder a información del sistema).
- Java.
- Python.
- C++

Grupo no.	**:**	**13**
Tema	**:**	**Implementación de una Máquina Virtual Simple**
Objetivo	**:**	Desarrollar una máquina virtual simple que permita **interpretar y ejecutar un conjunto básico de instrucciones**, simulando el funcionamiento de un procesador mediante el uso de memoria, registros y un contador de programa, con el propósito de comprender el ciclo básico de ejecución de instrucciones y la interacción entre hardware y software.
Descripción	**:**	**Dificultad:** Media | **Lenguaje:** Java | **Concepto del sistema operativo:** Máquina virtual y ejecución de instrucciones El proyecto consiste en desarrollar una máquina virtual sencilla en Java capaz de cargar e interpretar un pequeño programa compuesto por instrucciones previamente definidas. La máquina virtual simulará algunos componentes básicos de un procesador, como: • Memoria. • Acumulador. • Contador de programa (PC). • Registro de instrucción (IR). • Unidad de control. • Registro de estado, opcional. El sistema deberá ejecutar las instrucciones siguiendo el ciclo: FETCH → DECODE → EXECUTE Es decir: 1. Fetch: obtener la instrucción desde la memoria. 2. Decode: identificar qué operación debe realizar. 3. Execute: ejecutar la instrucción. Para mantener el proyecto en un nivel medio, se utilizará un conjunto reducido de instrucciones, evitando características avanzadas de una máquina virtual completa. **Funciones principales** • Simulación de memoria: Implementar una memoria virtual sencilla • Registros básicos • Conjunto de instrucciones: Implementar como mínimo PC: contador de programa. IR: registro de instrucción, ACC: acumulador. • Conjunto de instrucciones • Ciclo de ejecución • Carga de programas • Visualización de la ejecución • Manejo básico de errores • Interfaz

TREGABLE 1: Documento técnico y diseño del simulador

Valor sugerido: 40 %

Deberán presentar un documento que explique cómo funcionará el simulador de controladores de dispositivos.

Contenido mínimo

1. **Portada**
2. **Introducción**
3. **Planteamiento del problema**
4. **Objetivo general y objetivos específicos**
5. **Justificación**
6. Marco teórico
7. Diseño de la arquitectura
   - Memoria virtual.
   - Registros.
   - Acumulador (ACC).
   - Contador de programa (PC).
   - Registro de instrucción (IR).
   - Unidad de control.
   - Registro de estado, si se implementa.
   - Y mas….
8. Diseño del conjunto de instrucciones

Instrucción	Función
LOAD	Cargar un valor desde memoria
STORE	Almacenar un valor en memoria
ADD	Sumar
SUB	Restar
JMP	Realizar un salto
JZ	Salto si el resultado es cero
INPUT	Entrada de datos
OUTPUT	Salida de datos
HALT	Finalizar ejecución
Y mas …..	

9. Diseño del ciclo de instrucción
   Explicar:
   FETCH → DECODE → EXECUTE
   mostrando qué ocurre con el PC, IR, ACC y memoria en cada etapa.

10. Diseño de la memoria
    - Tamaño.
    - Direcciones.
    - Forma de almacenar instrucciones.
    - Forma de almacenar datos.
    - Y mas….
11. Lenguaje de instrucciones
    Definir cómo escribirá el usuario los programas que ejecutará la máquina virtual.
    Ejemplo:
    LOAD 10
    ADD 11

STORE 20
OUTPUT
HALT

12. Diagramas
    - Diagrama de arquitectura.
    - Diagrama de flujo de ejecución.
    - Diagrama del ciclo Fetch-Decode-Execute.
    - Diagrama de funcionamiento general.
    - Y mas….
13. Casos de prueba
    Diseñar al menos 5 programas de prueba, por ejemplo:
    - Suma.
    - Resta.
    - Almacenamiento.
    - Condicional.
    - Salto.
    - Y mas….
14. **Resultados esperados**
15. **Conclusiones**
16. **Bibliografía**

Producto a entregar

Documento impreso + diagramas + diseño del conjunto de instrucciones + casos de prueba.

Dispositivos mínimos

Para mantener la dificultad en **media**, recomiendo exigir solamente **tres dispositivos virtuales**:

- **Teclado**
- **Disco**
- **Impresora**

Cada dispositivo deberá tener su propio controlador simulado.

ENTREGABLE 2. Implementación, pruebas y defensa

Valor sugerido: 60 puntos

En este entregable los estudiantes deberán presentar la **máquina virtual funcionando en Java**.

Funcionalidades mínimas

1. Memoria virtual simulada

Debe permitir almacenar:

- Instrucciones.
- Datos.
- Direcciones de memoria.

2. Registros

Implementar como mínimo:

```
PC  → Program Counter
IR  → Instruction Register
ACC → Acumulador
```

Opcionalmente:

```
FLAGS → Registro de estado
```

3. Ciclo de ejecución

La máquina deberá ejecutar:

```
FETCH
   ↓
DECODE
   ↓
EXECUTE
   ↓
ACTUALIZAR PC
   ↓
SIGUIENTE INSTRUCCIÓN
```

4. Conjunto de instrucciones

Implementar las instrucciones definidas en el primer entregable.

Se recomienda que sean **al menos 8 instrucciones**.

5. Carga de programas

El usuario deberá poder introducir o cargar un programa en el lenguaje de instrucciones definido por el grupo.

Ejemplo:

```
LOAD 10
ADD 11
STORE 20
```

```
OUTPUT
HALT
```

6. Ejecución paso a paso

El sistema debe permitir observar cómo cambia la máquina después de cada instrucción.

Ejemplo:

```
================================
ESTADO DE LA MÁQUINA
================================

PC  : 03
IR  : STORE 20
ACC : 25

MEMORIA[20] : 25

INSTRUCCIÓN EJECUTADA: STORE
================================
```

7. Ejecución automática

También debe existir una opción para ejecutar todo el programa automáticamente.

8. Manejo de errores

Debe detectar situaciones como:

- Instrucción inexistente.
- Dirección de memoria inválida.
- Registro no válido.
- Programa sin HALT.
- Error de sintaxis.
- Acceso fuera del límite de memoria.

9. Visualización

La aplicación deberá mostrar como mínimo:

- Estado de los registros.
- Memoria.
- Instrucción actual.
- PC.
- Resultado de la operación.
- Estado de ejecución.

Puede realizarse mediante **consola o interfaz gráfica**.

10. Pruebas

Ejecutar los casos definidos en el Entregable 1 y presentar evidencias.

Prueba	Resultado esperado	Resultado obtenido	Estado
Suma	ACC = 30	ACC = 30	✓

Prueba	Resultado esperado	Resultado obtenido	Estado
Resta	ACC = 20	ACC = 20	✓
STORE	MEM[20] = 30	MEM[20] = 30	✓
Salto	Cambia PC	Correcto	✓
HALT	Finaliza	Correcto	✓

Documentación final

El segundo entregable deberá incluir:

- Código fuente en Java.
- Manual de usuario.
- Manual técnico.
- Evidencias de funcionamiento.
- Resultados de las pruebas.
- Conclusiones.
- Recomendaciones.

Defensa del proyecto

Cada grupo realizará una demostración de **15–20 minutos**.

Deberán demostrar:

1. Cómo está construida la máquina virtual.
2. Cómo funciona la memoria.
3. Cómo funcionan PC, IR y ACC.
4. Cómo se carga un programa.
5. Cómo se realiza el **FETCH**.
6. Cómo se realiza el **DECODE**.
7. Cómo se realiza el **EXECUTE**.
8. Ejecución paso a paso.
9. Ejecución automática.
10. Manejo de un error.
11. Resultado final del programa.

RÚBRICA DE EVALUACIÓN – MÁQUINA VIRTUAL SIMPLE

Entregable 1: Diseño y arquitectura de la máquina virtual

Valor: 40 puntos

Criterio	Excelente	Bueno	Básico	Deficiente	Puntos
1. Documento técnico	Presenta introducción, problema, objetivos, justificación y marco teórico completos y coherentes.	Presenta la mayoría de los elementos con pequeñas deficiencias.	Presenta información limitada.	Información incompleta o poco relacionada con el proyecto.	**5**
2. Diseño de la arquitectura	Define claramente memoria, registros, PC, IR, ACC y unidad de control, mostrando su relación.	Define los componentes principales correctamente.	Define algunos componentes, pero con poca profundidad.	Presenta un diseño incompleto o incorrecto.	**8**
3. Diseño del conjunto de instrucciones	Define claramente al menos 8 instrucciones, sintaxis y funcionamiento.	Define las instrucciones principales correctamente.	Presenta instrucciones limitadas o poco explicadas.	No existe un conjunto de instrucciones definido.	**7**
4. Ciclo Fetch–Decode–Execute	Explica detalladamente cada etapa y su relación con PC, IR, ACC y memoria.	Explica correctamente las tres etapas.	La explicación es superficial.	No comprende adecuadamente el ciclo.	**6**
5. Diagramas	Los diagramas son completos, claros y representan correctamente el funcionamiento.	Los diagramas son adecuados con pequeños errores.	Presenta diagramas simples o incompletos.	No presenta diagramas o son incorrectos.	**5**
6. Casos de prueba	Diseña 5 o más casos de prueba claros con resultados esperados.	Presenta casos de prueba suficientes.	Presenta pocos casos de prueba.	No presenta casos de prueba.	**5**
7. Organización y presentación	Documento ordenado, formal, coherente y sin errores significativos.	Buena presentación con algunos errores.	Presentación aceptable.	Documento desorganizado.	**4**
TOTAL					**40**

Entregable 2: Implementación, pruebas y defensa

Valor: 60 puntos

Criterio	Excelente	Bueno	Básico	Deficiente	Puntos
1. Implementación de memoria	Simula correctamente memoria para instrucciones y datos, con control de direcciones.	La memoria funciona correctamente con pequeños errores.	Funciona parcialmente.	No implementa correctamente la memoria.	**8**
2. Implementación de registros	PC, IR y ACC funcionan y se actualizan correctamente durante la ejecución.	Los registros principales funcionan correctamente.	Presenta algunos errores.	Los registros no funcionan adecuadamente.	**7**
3. Conjunto de instrucciones	Implementa correctamente 8 o más instrucciones.	Implementa la mayoría de las instrucciones.	Implementa un conjunto limitado.	Las instrucciones presentan errores graves.	**10**
4. Ciclo Fetch–Decode–Execute	El ciclo funciona correctamente y puede observarse paso a paso.	El ciclo funciona correctamente.	Funciona parcialmente.	No implementa correctamente el ciclo.	**10**
5. Carga y ejecución de programas	Permite cargar, interpretar y ejecutar programas correctamente.	Permite ejecutar programas con pequeños inconvenientes.	La ejecución es limitada.	No logra ejecutar programas correctamente.	**6**
6. Manejo de errores	Detecta y controla diferentes errores de instrucciones, memoria y ejecución.	Maneja los errores principales.	Maneja algunos errores.	No existe manejo de errores.	**4**
7. Visualización	Muestra claramente memoria, registros, instrucción actual y resultados.	Presenta adecuadamente el estado de la máquina.	La información mostrada es limitada.	La visualización es insuficiente.	**4**
8. Pruebas y evidencias	Ejecuta todos los casos de prueba y presenta evidencias claras de los resultados.	Ejecuta la mayoría de las pruebas.	Presenta pocas pruebas.	No demuestra el funcionamiento.	**4**
9. Código y documentación	Código organizado, modular, comentado y acompañado de manual técnico y de usuario.	Código organizado y documentación adecuada.	Código poco organizado o documentación limitada.	Código desorganizado y sin documentación.	**4**
10. Defensa del proyecto	Explica con dominio la arquitectura, funcionamiento y decisiones técnicas; responde correctamente las preguntas.	Explica adecuadamente el proyecto.	Presenta dificultades para explicar algunos aspectos.	No demuestra dominio del proyecto.	**3**
TOTAL					**60**

Escala de valoración

Puedes utilizar esta escala para cada criterio:

Nivel	Descripción	Equivalencia
Excelente	Cumple completamente, con precisión y demuestra dominio del tema.	90–100 %
Bueno	Cumple correctamente, aunque presenta pequeños errores.	80–89 %
Básico	Cumple parcialmente y requiere mejoras.	60–79 %
Deficiente	No cumple con los requisitos establecidos.	0–59 %

Distribución final

Evaluación	Puntaje
Entregable 1 – Diseño y arquitectura	**40 puntos**
Entregable 2 – Implementación, pruebas y defensa	**60 puntos**
TOTAL	**100 puntos**

Criterio clave del proyecto: además de que el programa funcione, el estudiante debe ser capaz de **explicar cómo la máquina virtual ejecuta una instrucción desde que la obtiene de memoria hasta que produce el resultado**, utilizando correctamente los conceptos de **PC, IR, ACC, memoria y ciclo Fetch–Decode–Execute**.